Introduction to Python

Week 1

What is Python?

- It is a programming language
  - A Python script is readable for human beings
  - A Python script is not readable for computers: a python script must be translated, before a computer can read and execute it
- Python is translated in two steps:
  1. The Python script is translated line by line into so-called byte code
  2. This byte-code is translated into machine-code
- Writing code must be precise. Ambiguities, that would be correctly interpreted by humans, will lead to errors in Python
- `We use a so-called monospaced font for code on these slides. This means that all characters take an equal amount of space. Using a monospaced font for code is a common practice`

Why Python?

- Python forces you to work in a structured way, and this is one of the reasons it is used in places, like universities, where programming is taught
- Python is hot as it is used a lot in (packages for) artificial intelligence/machine learning
- Some trade-offs:
  - Python is relatively slow
  - Python is very expandable
    - It is easy to let Python cooperate with programs written in faster (e.g. compiled) languages like C, C++, FORTRAN and RUST
    - There are a lot of packages (extensions) available for Python, and some have faster programming languages under the hood

Program for Today

1. Organization of the Course
2. Python

Organization of the course

- Instructors
- Topics
- Tools
- Communication
- Study advice
- Pythonanywhere
- Assesment

Instructors

- **Mathijs Janssen** (course coordinator, lecturer, instructor for tutorial groups 5 and 10)
- **Reza Armakan** (instructor for tutorial groups 11, 12, and 13)
- **Alessandro Capra** (instructor for tutorial groups 6, 7, 8 and 9)
- **Oliver Feltham** (instructor for tutorial groups 1, 2, 3 and 4)
- **Ieva Rudzite** and **Catalin Zaharia (**Discussions page and email: intropython-eb@uva.nl)

Communication

- Central to our communication is the discussion board on Canvas
  - If you have questions about Python, you can ask them via the discussion board
  - They can be answered and discussed by your fellow students, but your teachers will regularly check the discussion board and solve any loose ends
- Questions that are more personal can be asked via the mail function of canvas

Learning goals

- Understanding code (Tutorials, lectures), this is exam material
- Writing code (Tutorials, lectures), this is homework and can lead to a higher grade
- Getting an idea about Python in the real world, possible guest lectures

Tools

- Lectures (beginning of the week)
  - presentation of material about the python language that you need to know for the exam and that you can use for the programming exercises
  - Lectures are supported by lecture slides and Jupyter notebooks
  - Every week, you will get a set of personalized questions and a Python program that you can use to send your answers to our server and that server will tell you immediately whether your answer produce the right results, these questions can be quite complicated
  - You can send in solutions as often as you want, till the deadline Sunday night 23:59
  - Your answers will be tested automatically. Your code will be applied to several testsets. If a program produces the right result, you get a point, if it doesn't you won't get a point for that question
  - At the beginning of the next week, you will get feedback that tells you whether your solution worked correctly and also will give an example how to solve these exercises with the material, you had to study so far
  - These feedback solutions are also material to learn for the exam
- DataCamp: a website with videos about the course material. The website will present you with questions, and instant feedback to your answers

Tools

- Tutorials: here you can work on your homework and discuss problems with your fellow students and the tutors
- Homework exercises can be more difficult than exam questions and exam questions will be more difficult/complex than DataCamp questions
- Q&A sessions (at the end of the week):
  - Tying loose ends
    - Material we could not deal with in the first lecture of the week
    - Questions you asked, during lectures, during tutorials and on the discussion board
- The internet, has tons of well-done material on Python, some examples:
  - https://www.w3schools.com/python/ (the level is comparable to DataCamp, but a bit more extensive)
  - https://www.StackOverflow.com (the holy grail if you need help writing programs)
  - https://realpython.com/ (good tutorials, but not all are free. If you want to become very good, maybe spend some money here)

Running Python

- There are a lot of options, but we will only mention a few
  - Notebooks can be run in the browser on Colab (https://colab.research.google.com/), If you want to be fancier have a look at the Anaconda project (https://anaconda.org/anaconda/anaconda-project)
  - We advise you to use *Pythonanywhere for writing and testing your Python programs for the homework*
    - *Pythonanywhere* runs in your browser
    - Pythonanywhere is very easy to set up and use and your teachers can help you with it
- You are free to use other systems to run Python, but we cannot always offer help. There are just too many options

Study advice

- Before the lecture, study the DataCamp material
- After the lecture have a good look again at the notebook and the slides for that week
- Play with the notebooks. Run the code, change the code and run it again
- Come to the tutorials and do your homework exercises
- If you don't understand something, google for a solution, and ask a question on the discussion board

Assessment

- Midterm and Final exam
- Multiple choice
  - Questions will test your ability to understand code
    - Examples:
      - Which code fragment will print the following output
      - What will be printed by the following code fragment
      - Which code fragment prints the same output as the following code fragment
      - Of the following code fragments, one prints an output that differs from the other fragments
      - Which of the following code fragments solves the following problem correctly
    - Options can include:
      - None of the above
      - Both
      - All
      - An error
  - You're allowed to use one (double-sided) A4-cheatsheet during the exam
    - Making a good cheat sheet will help you to learn the material for the exam, and maybe you won't need it during the exam anymore
- Final exam grade must be at least 5.0
- Total grade must be at least 5.5

Python

- Logical and physical lines of code
- Comments
- Objects
- Operators

Logical lines of codes

- A Python program consists of
  - Logical lines of code
- Simple statements consist of one logical line of code

```
a = 1
```

- Compound statements consist of several logical lines of code

```
if a > 0:
    print("a is positive")
else:
    print("a zero or negative")
```

Physical lines of code

- In the previous slide every logical line was one physical line
- Two or more physical lines can be joined into one logical line with the help of backward slashes, this is called explicit line joining:

```
a = 'Two or more physical lines can be joined' +\
    ' into one logical line with the help of backward slashes'
```

- Sometimes one can use parts of a logical line inside a pair of parentheses, or square brackets, or curly brackets, to join physical lines, this is called implicit joining

```
a = ['Sometimes one can use parts of a logical line inside a',
     'pair of parentheses, or square brackets, or curly brackets, ',
     'to join physical lines,this is called implicite joining ']
```

- These examples of joining are easy to follow, but the precise rules of the explicit and implicit joining are a bit more complicated. However, in the exam material you can always assume the rules for joining are applied correctly

Comments

```
# This whole physical line is a comment
a = 1 # Everything after the hashtag (#) is a comment
# This is a
# multiline comment
```

Objects

- Every object has exactly one unique id that cannot be changed
- Every object has exactly one type that cannot be changed
- Every objects has exactly one value
- The type of an object defines which type of value can be stored in the object
- The type of an object defines whether a value is mutable or is immutable
- The value of a mutable objects can be changed, the value of an immutable object cannot be changed
- An Object can have zero, one or more names bound/referring to it
  - A name can only refer to one object, at any time
  - Names can be unbound from an object and bound to another object any time

Objects

- An object can be deleted
  - By using a `del` statement: `del(name_of_object)`
  - When an object is not referred to, not by a name or anything else, it can no longer be used in a program and Python's garbage collection will automatically remove the object
  - Objects created in a program [function], will only exist during the lifetime of the program [function], and will be automatically deleted when the program [function] ends
- Everything in Python is an object

Assignment statements (I)

- A common way to create an object is using an assignment statement, e.g.

```
name_1 = 300
```

- To follow exactly what Python will do, it is best to read these assignment statements from right to left:
  - Python creates a new object with value 300, type int, and a unique id
  - Python infers the object type from the value on the right side of the assignment statement. In this case the type will be `int` (integer)
  - Python binds the name on the left side of the assignment statement `name_1` to this newly created object
- We have created an object with the name `name_1` and we can now display the value, the type and the id of the object:

```
print (name_1)
print (type(name_1))
print (id(name_1))
```

Using assignment statements to create objects(II)

- Let's now create a second object with value 300 and type int

```
name_2 = 300
```

- We can show, that `name_1`, and `name_2` refer to objects that have the same value, and the same type, but have different id's

```
print(name_1 == name_2)
print(type(name_1) == type(name_2))
print(id(name_1) != id(name_2))
```

or shorter:

```
print(name_1 not is name_2)
```

Using assignment statements to create objects(III)

- You can also create an object and give it two (or more) names:

```
name_3 = 300
name_4 = name_3
name_5 = name_3
name_6 = name_4
```

or shorter:

```
name_3 = name_4 = name_5 = name_6 = 300
```

- We can show, that `name_3`, that `name_4`, that `name_5`, and `name_6` all refer to the same object, and have the same value and type:

```
print(name_3 == name_4 == name_5 == name_6)
print(type(name_3) == type(name_4) == type(name_5) == type(name_6))
print(name_3 is name_4 is name_5 is name_6)
```

Types of objects

- Integer

```
var_1 = 123
print(type(var_1) == int)
```

- Floating point number

```
var_1 = 123.3
print(type(var_1) == float)
```

- String

```
var_1 = '123'
print(type(var_1) == str)
```

- Boolean

```
var_1 = True
print(type(var_1) == bool)
```

Types of objects

- Tuple

```
var_1 = ('123', 123)
print(type(var_1) == tuple)
```

- List

```
var_1 = [123, '123']
print(type(var_1) == list)
```

- Dictionary

```
var_1 = {'key_1': 'elem_1', 'key_2': 'elem_2'}
print(type(var_1) == dict)
```

- Set

```
var_1 = {123, '123'}
print(type(var_1) == set)
```

Question

- Which of the following statements is correct:
  I. Two objects can have different values, while having the same type
  II. Two objects can have the same value, while having different types

a. I is correct
b. II is correct
c. I and II are both correct
d. Neither is correct

Answer

- Which of the following statements is correct:
  I. Two objects can have different values, while having the same type
  II. Two objects can have the same value, while having different types

a. I is correct
b. II is correct
c. I and II are both correct
d. Neither is correct

I is correct for example:

```
a = 1
b = 2
```

II is not correct, it is not possible to create 2 objects, with names `a` and `b` for which:

`print (a == b)` gives True
`print (type(a) == type(b)` gives False this

Names of objects

- Names of objects can consist of (capitalized) letters, digits, and underscores (_)
- Names of objects can start with (capitalized) letters, or an underscore
- Names of objects are case sensitive, e.g. `name_1` and `Name_1` are different names
- These are the minimal technical rules you have to abide too, if you don't follow them, Python throws an error
- Organizations where you work can have extra rules
- A common followed list of rules, can be found in the PEP 8 – Style Guide for Python Code (https://peps.python.org/pep-0008/)
- In this course we only follow the minimal technical rules

Arithmetic Operators

- `a + b`, `a - b`, `a * b`
- `a / b`, `a // b`
  - a / b results in a float
  - a // b results in the highest integer that is smaller or equal to the result of a division
- `a % b` gives the remainder of the integer division
- `a ** b` exponentiation
  - NB ^ (what is used in excel for exponentiation) is also a Python-operator but does something entirely different
- Question: What is the result of `(a//b)*b + a%b`

Arithmetic Operators

- `a + b`, `a - b`, `a * b`
- `a / b`, `a // b`
  - a / b results in a float
  - a // b results in the highest integer that is smaller or equal to the result of a division
- `a % b` gives the remainder of the integer division
- `a ** b` exponentiation
  - NB ^ (what is used in excel for exponentiation) is also a Python-operator but does something entirely different
- Question: What is the result of `(a//b)*b + a%b`
- Answer: `a`

Comparison operators

- `a == b`, `a != b`, `a > b`, `a < b`, `a >= b`, `a <= b`
- Always lead to True or False
- Comparing floats can be tricky: `1.1 + 2.2` according to Python is not equal to `3.3`. This is because floating point numbers are not precise. Computers use a binary system and that means that floats are not stored as precise as you maybe expect. For homeworkexercises we have built-in a tolerance so that when your answer is close enough you get a full point
- If you are interested in how to solve this problem with floating points, have a look at the `isclose` function or the `Decimal` type, this is not part of the exam
- Comparing strings, tuples and lists all work the same. Elements are compared from left to right, and the first elements that differs decides the result. If no element differs and the structures are of equal length, the structures are seen as equal. If no element differs and the structures are of different length, the longest structure is seen as larger

Boolean operators

- `x and y`: True if both are True, otherwise False
- `x or y`: False if both are False, otherwise True
- `not x`: True if x is False, False if x is True
  - NB `&&` and `||` (which are used in some programming languages for `and` and `or`) are also a Python-operators but do something entirely different

Mutable vs immutable

- List (mutable):

```
l1 = [1,2,3]
print(l1)   # Onscreen: [1, 2, 3]
l1[2] = 4
print(l1)   # Onscreen: [1, 2, 4]
```

- Tuple (immutable):

```
t1 = (1,2,3)
print(t1)   # Onscreen: (1, 2, 3)
t1[2] = 4
#  Onscreen: TypeError Traceback (most recent call last)
Input In [2], in <cell line: 1>()
----> 1 t1[2] = 4
TypeError: 'tuple' object does not support item assignment
```

Question

- We know objects with a string value are immutable
- Why then does the following code work

```
box_1 = 'content'
print(box_1)
box_1 = 'new content'
print(box_1)
```

Answer

- We know objects with a string value are immutable
- Why then does the following code work

```
box_1 = 'content'
print(box_1)
box_1 = 'new content'
print(box_1)
```

- Answer: we didn't change the object, we created a new object and gave it an old name

```
box_1 = 'content'
print(box_1)
old_id = id(box_1)
box_1 = 'new content'
print(box_1)
new_id = id(box_1)
print(old_id == new_id)
```

An example with a string (immutable)

`name_1 = 's1'` Python creates an object with the value `'s1'`, and binds the name `name_1` to that object.
We have now one object with the string value `'s1'` and with the name `name_1`

`name_2 = name_1` To the object with value `'s1'` and with the name `name_1`, Python binds a second name `name_2`.
We have now one object with the value `'s1'`, and with two names `name_1` and `name_2`

`name_1 = 's2'` From the object with value `'s1'` the name `name_1` is removed.
Python **creates** a new object with the value `'s2'` and binds the name `name_1` to this new object.
We have now one object with the value `'s2'` and with the name `name_1`,
and we have one object with the value `'s1'` and with the name `name_2`

`name_2 = 's3'` From the object with the value `'s1'` the name `name_2` is removed.
If as is the case here, this is the only reference left to that object, Python's garbage collection will remove this object.
We will now have one object with value `'s2'` and with the name `name_1`

An example with a list (mutable)

```
l1 = ['s1', 's2']
```

Python creates two objects, one with value `'s1'` and another one with value `'s2'`.
Next, Python creates an object of type list that refers to these two objects and binds the name `l1` to that list.
(The objects containing `'s1'` and `'s2'` have no names, but have id's and can be referred to.)

```
l2 = l1
```

To the list referring to the two string objects, Python binds a second name `l2`.

```
l1[0] = 's3'
```

Python creates a new object with the value `'s3'`.
Next, Python **mutates** the list, with the names `l1` and `l2`. The first object referred to by that list (with the names `l1` and `l2)` is no longer the object with value `'s1'`, but the object with value `'s3'`, while the second object referred to by that list (with the names `l1` and `l2)` is still the object with the value `'s2'`.

```
l2 = ['s4', 's5']
```

From the first list the name `l2` is removed.
As that list is still referenced by the name `l1`, the list will not be removed by Python's garbage collection.
Next, Python creates two objects, one object with the value `'s4'` and another object with the value `'s5'`.
Now Python creates a list referring to these two objects and binds the name `l2` to it.

Indexing

- `l1 = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']`
- The first element, seen from the left, is 0, the second is 1, and the last is `len(l1)-1`
- The first element, seen from the right, is -1, the second is -2, and the last is `-len(l1)`
- `print(l1[0] == l1[-len(l1)] )`

Slicing (I)

- Slicing creates a **new** object made consisting of elements of another object
- Slicing can be used with objects of type list, string and tuple and the result will be an object of the same type
- Syntax: `new_list = existing_list[start:end:step_size]`
- `Start`, `end`, and `step_size` are parameters of the slicing operation
- Start is inclusive, and tells Python with which index to start the slicing
- End is not inclusive, and tells Python with which index to stop the slicing (this could be beyond the highest or lowest index value)
- The first element is 0, the second is 1, and the last is `len(existing_list)`
- `You can also use -1 for the last element and -len(existing_list) for the first`
- `Step_size`
- `len` is a built_in function that gives you the number of elements when applied on a list, string or tuple

Slicing (I): examples when stepsize >= 1

- Recall: `new_list = existing_list[start:end:step_size]`
- `l1 = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']`
- `print(l1[0:6:2])`
- `print(l1[1:6:1] == l1[1:6])`
- `print(l1[1:6:1] == l1[1:6:])`
- `print(l1[0:6:2] == l1[:6:2])`
- `print(l1[2:len(l1):2] == l1[2::2])`
- `print(l1[0:len(l1):1] == l1[::])`

Question: what is the result of the following program?

- ```
  l1 = [1, 2, 3, 4]
  l2 = l1
  l3 = l1[::]
  l1[-1] = 5
  print(l1 == l2 , l2 == l3)
  ```
- And why?
  ```

Question: what is the result of the following program?

- ```
  l1 = [1, 2, 3, 4]
  l2 = l1
  l3 = l1[::]
  l1[-1] = 5
  print(l1 == l2 , l2 == l3)
  ```
- Answer: `True False`
- Look at the definition of slicing: Slicing creates a **new** object
  ```

Slicing (II): examples when stepsize >=1

- Recall: `new_list = existing_list[start:end:step_size]`
- `l1 = [1,2,3,4,5,6,7,8]`
  - `print(l1[7])  # You ask for the 8th element`
    `# Onscreen: 8`
  - `print(l1[-1]) # You ask for the last element`
    `# Onscreen: 8`
  - `print(l1[-5:5])`
    `# Onscreen: [4, 5]`
  - `print(l1[:-5])`
    `# Onscreen: [1, 2, 3]`

Slicing (III): examples with negative steps

- **While with positive steps you go from the start to the right, with a negative step you go from the end to the left**
- `l1 = [1,2,3,4,5,6,7,8]`
  - `print(l1[5:1:-2]) # Onscreen: [6, 4]`
  - `print(l1[5::-1]) # Onscreen: [6, 5, 4, 3, 2, 1]`
  - `print(l1[:1:-1]) # Onscreen: [8, 7, 6, 5, 4, 3]`
  - `print(l1[1:5:-2]) # Onscreen: []`
    - **Python will check whether you are already past the end index, it works similar as** `print(l1[5:1:2]) # Onscreen: []`

Question: What will be printed?

- `l1 = [1, 2, 3, 4]`
- `print(l1[::-1])`

Question: What will be printed?

- `l1 = [1, 2, 3, 4]`
- `print(l1[::-1])`
- Answer: `[4, 3, 2, 1]`

Slicing with strings and tuples

- Slicing can also be done with strings and tuples
- ```
  t1 = (1, 2, 3)
  print(t1[1:]) # Onscreen: (2, 3)
  ```
- ```
  s1 = 'uva Amsterdam'
  print(s1[5:-3:2]) # Onscreen: mtr
  ```

Changing a slice of a list

- Changing a list:

```
l1 = [1, 2, 3, 4, 5, 6]
l1[1:4] = l1[1:4][::-1]
print(l1) # Onscreen: [1, 4, 3, 2, 5, 6]
```

- Tricky:

```
l1 = [1, 2, 3, 4, 5, 6]
l1[1:2] = [1, 1]
print(l1) # Onscreen: [1, 1, 1, 3, 4, 5, 6]
```

vs

```
l1 = [1, 2, 3, 4, 5, 6]
l1[1] = [1, 1]
print(l1) # Onscreen: [1, [1, 1], 3, 4, 5, 6]
```

'Changing' a slice of a string or a tupple

- 'changing' a string:
  - ```
    s1 = 'uva Amxterdam'
    s1[6] = 's'      # Gives an error
    ```
  - ```
    s1 = 'uva Amxterdam'
    s1 = s1[:6] + 's' + s1[7:]
    print(s1)        # works
    ```
- 'changing' a tuple:
  - ```
    t1 = (1, 4, 3)
    t1[1] = 2        # Gives an error
    ```
  - ```
    t1 = (1, 4, 3)
    t1 = t1[:1] + (2,) + t1[2:]
    print(t1)        # works
    ```

Range

- `range`*(start, stop, step*)
- Range function is used to generate a series of numbers. The start, stop and step parameters work similar as in list slicing, but as you will see the defaults work a bit different.

```
x = list(range(6))
print(x)     # Onscreen: [0, 1, 2, 3, 4, 5]

x = list(range(3,6))
print(x)     # Onscreen: [3, 4, 5]

x = list(range(3,6,2))
print(x)     # Onscreen: [3, 5]
```

Functions

- We often want to (re)use pieces of code
- Pieces of code can be wrapped into a function, given a name, and then re-used by referencing that name
- We can pass input to a function, these values are called arguments
- Functions can create output, we say functions can return a value

```
def size(length, width):
    return length * width
print(size (2,3)) # Onscreen: 6
```

- Python comes with a lot of built-in functions, we already saw:

```
print
id
len
type
del
```

- We don't much care how these built-in functions are written, we just want to know what it does (= how it turns inputs into outputs)
- We will write our own functions from Week 3 onwards

Objects

- Organizing your code in functions is a good idea, but organizing your code in objects may be even better
- Objects can encapsulate (≈ contain) functions and data
- Objects are always defined on the basis of a blueprint, and that blueprint is called a class, so you first have to define the class:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def size(self):
        return self.length * self.width
```

Objects

- ```
  class Rectangle:
      def __init__(self, length, width):
          self.length = length
          self.width = width
      def size(self):
          return self.length * self.width
  ```
- And now you can define your objects:

```
rectangle_1 = Rectangle(2,3)
rectangle_2 = Rectangle(3,3)
print(rectangle_1.size(), rectangle_2.size())
# Onscreen: 6 9
```
```

In Python, everything is an object

- Recall that every object has a type, a value, and zero or more names
- Functions that are encapsulated in objects are called methods
  - Depending on the type, an object has several methods
  - Methods can take arguments, and they also have access to the data inside the object
- For every datatype there is already a built-in class, that defines what you can do with an object of that type

In Python, everything is an object

- Sometimes, methods return a value
- Sometimes, methods change the object itself, but this is only possible if the object is mutable
  - If an object is immutable, the method applied on it must return a value as it cannot change that object itself
- Example of a method on an immutable object:
  - `s1='UVA Amsterdam'`
    `s1=s1.upper() or s1.upper()`
  - Which code is correct?
  - Answer: s1=s1.upper(), since the method upper() returns a new string. If you just do s1.upper() Python is ok with that, but the string itself cannot be changed. This error can be very difficult to find.
- For methods working on mutable objects, you have to know whether it changes the object or returns a value.
  - Example (l1 is a list): l1. sort() sorts the list itself
  - Example (l1 is a list): l1. index('a') returns the index of the first element in the list with value 'a'

Importing additional functionality from packages

- Python ecosystem = core language + standard library + third-party packages
- The ecosystem is enormous, there is no way to make it available by default
- That's why we need to install third-party packages separately, for example
- Even the standard library is large; it's functionality must be imported to be used
  - E.g. import math
- Importing is about making the contents of a module (= Python file) available in the namespace of our program

Example: Importing additional functionality from packages

- Example: how to access the randint function in the random module of the numpy package
- Via import, you make an object available to your program
- ```
  print(numpy.random.randint(1, 10)) # error
  ```
- ```
  import numpy
  print(numpy.random.randint(1, 10)) # works
  ```
- ```
  import numpy as np
  print(np.random.randint(1, 10))    # works
  print(numpy.random.randint(1, 10)) # error
  ```
- ```
  import numpy.random as rnd
  print(rnd.randint(1, 10))          # works
  print(numpy.random.randint(1, 10)) # error
  ```

Example: Importing additional functionality from packages

- ```
  from numpy.random import randint
  print(randint(1, 10))                  # works
  print(numpy.random.randint(1, 10)) # error
  ```
- ```
  from numpy.random import randint as
  give_me_a_random_integer
  print(give_me_a_random_integer(1, 10)) # works
  print(numpy.random.randint(1, 10))     # error
  ```
- NB!! Notice you can only give one name to an imported object, so if you use as to change the name, you no longer can use the original name